STOCHASTIC MODELING

MODULE 3 | LESSON 1

MODEL CALIBRATION: HESTON (1993)

Reading Time	130 minutes
Prior Knowledge	Fourier Transform, Characteristic Function (CF), Heston (1993), Lewis (2001)
Keywords	Stochastic volatility, Model calibration , Lewis (2001)

Finally, we have arrived to the crucial step of **model calibration**. Now that we have our bagpack full of tools, we can implement our first complex calibration. Let's calibrate our first stochastic volatility model!

In [1]:
```
# Let's start with some libraries that  we will need down the road
import numpy as np
from scipy.integrate import quad
```

1. The problem we usually encounter in practice

In [2]:
```
from IPython.display import VimeoVideo

# Bigger video
VimeoVideo("1121443520", h="3298dbabb7", width=700, height=450)
```

Out[2]:

Suppose you have just landed a job as a Junior Quant at an Investment Bank (IB). One common service that sell-side institutions such IB provide is the pricing of exotic derivatives for clients. Think of an Oil company that needs a tailor-made derivative to hedge their exposure to swings in Crude Oil prices. Or a Hedge fund that wants to build a position in derivatives (long or short) in a bunch of companies through a basket option.

After a few months at your job, you are ready to take over specific projects by yourself. A client has come to the Sales department big boss asking for an exotic derivative (e.g, an Asian option on S&P500). Sales guys do a great job in dealing with clients, but they ultimately need the Quant team to price the products for them. So, they contact your boss at the Quant team, which assigns this task to you.

How would you proceed?

1. Which **model** to use? Black-Scholes, Merton, Heston, ...?
2. Let's suppose Heston, $\rightarrow$ how do you get the **parameter values** of the model?
3. **Calibration!**: which maturities? calls & puts? moneyness?

$\rightarrow$ Trade-off between accuracy and generality (more on this later)

4. **Price** exotic instrument using, for example, Monte-Carlo methods.

5. Get final price for the client by adding **the bank's fee**

- If we price too high, client may run to a rival bank!
- If we price too low, bank will loose money $\rightarrow$ why? Hedging!

6. Hedging position!

As you can see, there are a lot of small decisions involved in the overall process, but **calibration** plays a central role. Without a proper fit of the model to market prices (i.e., properly capturing investor's expectations) the overall process is flawed.

2. Heston model: Pricing under Lewis (2001)

In order to calibrate a model there are a bunch of things that we should do and consider first. The first, is to obtain a (semi) closed-form solution to the price of the option under the model we are going to use, in this case Heston (1993). For that, in this lesson we are going to follow the Lewis (2001) approach, although things will be done analogously for other methods such as Carr-Madan.

2.1. Pricing in Heston under Lewis (2001)

Following Lewis (2001), the expression for the price of a Call option is defined by:

$$ C_0 = S_0 - \frac{\sqrt{S_0 K} e^{-rT}}{\pi} \int_{0}^{\infty} \mathbf{Re}[e^{izk} \varphi(z-i/2)] \frac{dz}{z^2+1/4} $$

Once we have this equation as a reference for pricing, we need:

- A characteristic function, $\varphi(z-i/2)$, for the Heston (1993) model.
- A way to numerically calculate the integral value $\rightarrow$ quadrature method (*quad* library in *scipy*).

In general, you can see that the problem will most often lie in unveiling the characteristic function for the process followed by the underlying asset. Let's then derive the Heston (1993) characteristic function.

2.2. Heston (1993) characteristic function (CF)

The most important ingredient of Fourier transform methods such as Lewis (2001) is knowledge of the characteristic function for the underlying process. Deriving the characteristic function of Heston (1993) is not as easy and straightforward as in the case of Black-Scholes. We will present here the closed-form expression; you can

check the original Heston (1993) paper or Gatheral (2006) to see the derivation of this characteristic function.

The characteristic function of the Heston (1993) model is given by:

$$ \ \begin{equation*} \varphi^{H} (u, T) = e^{H_1(u, T)+H_2(u,T)\nu_0} \end{equation*} $$
where

$$ \ \begin{equation*} H_1 (u, T) \equiv r_0 uiT + \frac{c_1}{\sigma_\nu^2}\Biggl\{ (\kappa_\nu - \rho \sigma_\nu ui+c_2) T - 2 log \left[\frac{1-c_3e^{c_2T}}{1-c_3} \right] \Biggl\} \end{equation*} $$

$$ \ \begin{equation*} H_2 (u, T) \equiv \frac{\kappa_\nu - \rho \sigma_\nu ui + c_2} {\sigma_\nu^2} \left[\frac{1-e^{c_2T}}{1-c_3e^{c_2T}} \right] \end{equation*} $$

$$ \ \begin{equation*} c_1 \equiv \kappa_\nu \theta_\nu \end{equation*} $$

$$ \ \begin{equation*} c_2 \equiv - \sqrt{(\rho \sigma_\nu ui - \kappa_\nu)^2 - \sigma_\nu^2(-ui-u^2) } \end{equation*} $$

$$ \ \begin{equation*} c_3 \equiv \frac{\kappa_\nu - \rho \sigma_\nu ui + c_2} {\kappa_\nu - \rho \sigma_\nu ui - c_2} \end{equation*} $$

As you can see, the derivation and closed-form expression for the characteristic function of the Heston model is not simple at all. Luckily for us, we can create a function in Python that simplifies its calculations every time:

In [3]:
```
def H93_char_func(u, T, r, kappa_v, theta_v, sigma_v, rho, v0):
    """Valuation of European call option in H93 model via Lewis (2001)
    Fourier-based approach: characteristic function.
    Parameter definitions see function BCC_call_value."""
    c1 = kappa_v * theta_v
    c2 = -np.sqrt(
        (rho * sigma_v * u * 1j - kappa_v) ** 2 - sigma_v**2 * (-u * 1j -
    )
    c3 = (kappa_v - rho * sigma_v * u * 1j + c2) / (
        kappa_v - rho * sigma_v * u * 1j - c2
    )
    H1 = r * u * 1j * T + (c1 / sigma_v**2) * (
        (kappa_v - rho * sigma_v * u * 1j + c2) * T
        - 2 * np.log((1 - c3 * np.exp(c2 * T)) / (1 - c3))
    )
    H2 = (
        (kappa_v - rho * sigma_v * u * 1j + c2)
        / sigma_v**2
        * ((1 - np.exp(c2 * T)) / (1 - c3 * np.exp(c2 * T)))
    )
```

```
    char_func_value = np.exp(H1 + H2 * v0)
    return char_func_value
```

2.3. Value of the integral in Lewis (2001)

Now that we have our characteristic function, let's move on to another important step in the pricing process: the value for the integral in Lewis (2001):

$$ \ \begin{equation*} C_0 = S_0 - \frac{\sqrt{S_0 K} e^{-rT}}{\pi} \int_{0}^{\infty} \mathbf{Re}[e^{izk} \varphi(z-i/2)] \frac{dz}{z^2+1/4} \end{equation*} $$

Obviously, the expression for the integral is the same one we used for Black-Scholes, but note that the expression for the characteristic function has changed.

In [4]:
```
def H93_int_func(u, S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0):
    """
    Fourier-based approach for Lewis (2001): Integration function.
    """
    char_func_value = H93_char_func(
        u - 1j * 0.5, T, r, kappa_v, theta_v, sigma_v, rho, v0
    )
    int_func_value = (
        1 / (u**2 + 0.25) * (np.exp(1j * u * np.log(S0 / K)) * char_func_
    )
    return int_func_value
```

2.4. Call option value under Lewis (2001)

Finally, we will need to numerically compute the value of the aforementioned integral to solve for the (semi) closed-form price of the Call option. As before, we will use the quadrature method (*quad*) included in the scipy package (https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.quad.html)

In [5]:
```
def H93_call_value(S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0):
    """Valuation of European call option in H93 model via Lewis (2001)

    Parameter definition:
    ==========
    S0: float
        initial stock/index level
    K: float
        strike price
    T: float
        time-to-maturity (for t=0)
    r: float
        constant risk-free short rate
    kappa_v: float
        mean-reversion factor
    theta_v: float
        long-run mean of variance
    sigma_v: float
        volatility of variance
```

```
    rho: float
        correlation between variance and stock/index level
    v0: float
        initial level of variance
    Returns
    =======
    call_value: float
        present value of European call option
    """
    int_value = quad(
        lambda u: H93_int_func(u, S0, K, T, r, kappa_v, theta_v, sigma_v,
        0,
        np.inf,
        limit=250,
    )[0]
    call_value = max(0, S0 - np.exp(-r * T) * np.sqrt(S0 * K) / np.pi * i
    return call_value
```

2.5. Pricing example

The most important use of the previous code is to have a closed-form expression for the price of the option. This closed-form expression will be crucial for the calibration process that we will undertake in the next step.

For now, we can simply check that the pricing function works by considering a specific Call option characteristics and certain given parameter values for Heston:

In [ ]:
```
# Option Parameters
S0 = 100.0
K = 100.0
T = 1.0
r = 0.02

# Heston(1993) Parameters
kappa_v = 1.5
theta_v = 0.02
sigma_v = 0.15
rho = 0.1
v0 = 0.01
```

In [ ]:
```
print(
    "Heston (1993) Call Option Value:   $%10.4f "
    % H93_call_value(S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0)
)
```

Of course, things are super simple if we had the model parameters to begin with. But that is no fun. In practice, we do not know almost anything about these parameter values (maybe, with experience, we can have educated guesses at best). So we have to calibrate the model to market prices to obtain them!

3. Heston Model Calibration

We will calibrate our Heston model by looking at market option prices. Hence, the first thing we need is options' market data to work with. Specifically, we are going to

calibrate our Heston model using market data for options on the EuroStoxx 50 index (Europe's 50 largest firms).

We will take the data for just one day, assuming this is the day at which our client demands the price for the exotic derivative. In practice, one would ideally calibrate to market data for a bunch of dates, in an exercise to explore the stability (or lack of thereof) of the parameter values. If our parameter values vary little across different calibrations, that's usually good news for the robustness of the price and the subsequent hedging process. If they vary a lot, we should be careful with these.

Let's start by importing some additional libraries needed:

In [ ]:
```
import pandas as pd
from scipy.optimize import brute, fmin
```

3.1. Gather Options' Market Data

Next, let's load the data on market prices:

In [ ]:
```
# Market Data from www.eurexchange.com
# as of September 30, 2014

h5 = pd.HDFStore(
    "option_data_wqu.h5", "r"
    )
data = h5["data"]  # European call & put option data (3 maturities)
h5.close()
S0 = 3225.93  # EURO STOXX 50 level September 30, 2014
```

In [ ]:
```
data.head()
```

Once you have the market data loaded, we are going to select the options that we want to be part of the calibration process.

We will select near ATM options. This is usually the case provided that data near ATM is sufficienlty rich, because as we move away from ATM there are some issues (such as liquidity) that may impact option prices that does not have to do directly with investors' expectation on the underlying asset.

In [ ]:
```
# Option Selection

tol = 0.02  # Tolerance level to select ATM options (percent around ITM/O
options = data[(np.abs(data["Strike"] - S0) / S0) < tol]
options = options.copy()
options["Date"] = pd.to_datetime(options["Date"])
options["Maturity"] = pd.to_datetime(options["Maturity"])
```

Then, we add time left until maturity and a constant risk-free rate:

In [ ]:
```
for row, option in options.iterrows():
    T = (option["Maturity"] - option["Date"]).days / 365.0
```

```
    options.loc[row, "T"] = T
    options.loc[row, "r"] = 0.02
```

In [ ]:
```
options
```

3.2. Calibration process

Now that we have the data, let's begin our calibration process. Apart from the previously defined functions (or, better said, building on those), we will need to define some additional functions to optimize our model parameters so that they match observed market data.

First, we will introduce a function that will evaluate the error the model makes with respect to observed data given certain parameters. As usual, we will rely on a **mean squared error (MSE) function**. We will also define some initial values for the calibration parameters:

In [ ]:
```
i = 0
min_MSE = 500
```

In [ ]:
```
def H93_error_function(p0):
    """Error function for parameter calibration via
    Lewis (2001) Fourier approach for Heston (1993).
    Parameters
    ==========
    kappa_v: float
        mean-reversion factor
    theta_v: float
        long-run mean of variance
    sigma_v: float
        volatility of variance
    rho: float
        correlation between variance and stock/index level
    v0: float
        initial, instantaneous variance
    Returns
    =======
    MSE: float
        mean squared error
    """
    global i, min_MSE
    kappa_v, theta_v, sigma_v, rho, v0 = p0
    if kappa_v < 0.0 or theta_v < 0.005 or sigma_v < 0.0 or rho < -1.0 or
        return 500.0
    if 2 * kappa_v * theta_v < sigma_v**2:
        return 500.0
    se = []
    for row, option in options.iterrows():
        model_value = H93_call_value(
            S0,
            option["Strike"],
            option["T"],
            option["r"],
            kappa_v,
            theta_v,
            sigma_v,
```

```
            rho,
            v0,
        )
        se.append((model_value - option["Call"]) ** 2)
    MSE = sum(se) / len(se)
    min_MSE = min(min_MSE, MSE)
    if i % 25 == 0:
        print("%4d |" % i, np.array(p0), "| %7.3f | %7.3f" % (MSE, min_MS
    i += 1
    return MSE
```

Note: Please beware that here we are going to calibrate the model only to Call options. It is a best practice, though, to do this for both Calls and Puts, because sometimes put-call parity in real life may not perfectly hold and we can gain richer calibrations considering both option types. The reason why Puts are not considered through this notes is that including them in the calibration process will be part of your workload for group projects and quizzes

Next, we will need a function that performs the **optimization process**. In other words, it optimizes the model parameters so as to minimize the error function with respect to market data. We will do this in 2 steps in order to look for faster convergence of the prices to market quotes. First, we will use the `brute` function of scipy (https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.brute.html), that allows the calibration to focus on most sensible ranges. Once these are declared, we can dig deeper into the specific regions and get the actual parameters more accurately with the `fmin` function (https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.fmin.html).

In [ ]:
```
def H93_calibration_full():
    """Calibrates Heston (1993) stochastic volatility model to market quo
    # First run with brute force
    # (scan sensible regions, for faster convergence)
    p0 = brute(
        H93_error_function,
        (
            (2.5, 10.6, 5.0),  # kappa_v
            (0.01, 0.041, 0.01),  # theta_v
            (0.05, 0.251, 0.1),  # sigma_v
            (-0.75, 0.01, 0.25),  # rho
            (0.01, 0.031, 0.01),  # v0
        ),
        finish=None,
    )

    # Second run with local, convex minimization
    # (we dig deeper where promising results)
    opt = fmin(
        H93_error_function, p0, xtol=0.000001, ftol=0.000001, maxiter=750
    )
    return opt
```

3.3. Results from Calibration

Now that we have all the necessary ingredients, let's see how our calibration algorithm performs. For that, given the way we structured things before, we just need to call our *H93_calibration_full()* function. This will give us each of the different outputs from calibration, including the values given to the different parameters in the model. Before running, please be aware of the time this algorithm will take!

In [ ]:
```
H93_calibration_full()
```

Now we have finally calibrated our parameters to market values.

The results from this calibration give us the following values for the parameters in the Heston (1993) model:

$\kappa_\nu = 5.047$

$\theta_\nu = 0.018$

$\sigma_\nu = 0.434$

$\rho = -0.447$

$\nu_0 = 0.027$

The next step will be simply using these parameters to price the option we want.

Code Task — Implement `H93_put_value_parity` (simple wrapper)

Goal: Implement a function `H93_put_value_parity(S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0)` that returns the **European put** price **using put–call parity** and the existing `H93_call_value` provided in this notebook.

Parity identity (no dividends): $$ \text{Put} = \text{Call} - S_0 + K \cdot \exp(-rT) $$

Requirements

1. Use the exact function signature shown below.
2. Call the existing `H93_call_value` to obtain the call price.
3. Compute the put via parity and **clamp tiny negatives to zero** (e.g., `max(0.0, value)`).
4. Return a Python `float`.
5. Do **not** import new libraries or change other cells.

Function signature to implement

```
def H93_put_value_parity(S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0):
    ...
```

In [ ]:
```
## 👷 Starter code

import numpy as np

def H93_put_value_parity(S0, K, T, r, kappa_v, theta_v, sigma_v, rho, v0)
    """
    Return the European put price using put-call parity and the existing

    Put = Call - S0 + K * exp(-r*T)

    Clamp tiny negative values to zero to avoid round-off artifacts.
    """
    # TODO: call H93_call_value(...) to get the call price
    # TODO: compute the parity RHS and clamp tiny negatives to zero
    # TODO: return a float
    raise NotImplementedError("Implement H93_put_value_parity as specifie
```

4. Conclusion

Well done! In this lesson, we have, first, used Fourier methods to price options using the Lewis (2001) approach for the Heston (1993) model, and second, developed a full calibration of the Heston (1993) model. If you understood the full process covered in this notebook, you are on the right track!

References

- Gatheral, Jim. *The Volatility Surface: A Practitioner's Guide*. John Wiley & Sons Inc., 2006.
- Heston, Steven L. "A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options." *The Review of Financial Studies*, vol. 6, no. 2, 1993, pp. 327-343.
- Hilpisch, Yves. *Derivatives Analytics with Python: Data Analysis, Models,Simulation, Calibration and Hedging*. John Wiley & Sons, 2015.
- Lewis, Alan L. "A Simple Option Formula for General Jump-Diffusion and Other Exponential Lévy Processes." 2001.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.